

3 Lecture 3: Bellman Optimality Equation, Value Iteration and Policy Iteration

3.1 Optimal Policy and Optimal State Value

Optimal Policy and Optimal State Value

State s 处的 state value 定义为该 state 处未来 (discounted) return 的期望值: $v_\pi(s) = \mathbb{E}[G_t | S_t = s]$. 对于不同的 policy 可以通过比较它们在某个 state 或全局的 state value 来比较它们的好坏.

Definition 3.1 (Better Policy). 对于两 policy π_1, π_2 , 若他们的 state value 有如下全局关系则称 policy π_1 好于 π_2 :

$$v_{\pi_1}(s) \geq v_{\pi_2}(s), \quad \forall s \in \mathcal{S}. \quad (3.1)$$

Definition 3.2 (Optimal Policy & Optimal State Value). 若 policy π^* 的 state value 有:

$$v_{\pi^*}(s) \geq v_\pi(s), \quad \forall s \in \mathcal{S}, \forall \pi, \quad (3.2)$$

则称 policy π^* 为 optimal policy. 称 π^* 的 state values 为 optimal state values, 并定义 $v^*(s) \triangleq \sup_\pi v_\pi(s)$. 在有限状态、有限动作且 $0 \leq \gamma < 1$ 的 MDP 中, 存在一个确定性最优策略 π^* 使得 $v_{\pi^*}(s) = v^*(s), \forall s \in \mathcal{S}$.

3.2 Bellman Optimality Equation

Bellman Optimality Equation

定义了 optimal policy 和 optimal state value 后自然而然引出了下列问题:

1. **Existence & Uniqueness:** optimal policy 是否存在? 若存在是否唯一?
2. **Stochasticity:** optimal policy 是确定性的 (deterministic) 还是随机性的 (stochastic)?
3. **Algorithm:** 如何得到 optimal policy 和 optimal state values?

分析 optimal policy 和 optimal state value 的重要工具是 **Bellman Optimality Equation (BOE)**. 与之前 Bellman Equation (BE) 思路相似, 通过求解 BOE 就可以得到 optimal policy 和 optimal state values. BE 形式如下:

$$v_\pi(s) = \underbrace{\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a)r}_{\text{mean of immediate rewards}} + \underbrace{\gamma \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a)v_\pi(s')}_{\text{mean of future rewards}} = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_\pi(s') \right], \quad \forall s \in \mathcal{S}. \quad (3.3)$$

Bellman Optimality Equation 不再像 Bellman Equation (3.3) 一样对给定 policy 的动作分布取期望, 而是在每个状态上调整 policy 使得 one-step look-ahead value (即当前 state value 展开一步 $G_t = R_t + \gamma G_{t+1}$) 最大:

$$v^*(s) = \max_\pi \sum_{a \in \mathcal{A}} \pi(a|s) \left(\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v^*(s') \right) \triangleq \max_\pi \sum_{a \in \mathcal{A}} \pi(a|s)q(s, a), \quad \forall s \in \mathcal{S}. \quad (3.4)$$

式 (3.4) 是 BOE 的 elementwise form, 与 BE 的 $\mathbf{v} = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}$ 形式相似, 其 matrix-vector form 如下:

$$\mathbf{v}^* = \max_\pi (\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}^*), \quad [\mathbf{r}_\pi]_s \triangleq \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s', r} p(s', r|s, a)r, \quad [\mathbf{P}_\pi]_{s, s'} \triangleq p(s'|s) \triangleq \sum_{a \in \mathcal{A}} \pi(a|s)p(s'|s, a). \quad (3.5)$$

Note 3.1. 当 reward 与下一 state s' 条件独立时有 $[\mathbf{r}_\pi]_s = \sum_a \pi(a|s) \sum_r p(r|s, a)r$.

根据定义 optimal policy 对所有 state 都有最优性, 因此 max 为逐状态取最大值, 矩阵形式中可以写进矩阵中:

$$\max_\pi \underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}}_{\max_\pi \mathbf{v}_\pi} = \begin{bmatrix} \max_\pi v_\pi(s_1) \\ \max_\pi v_\pi(s_2) \\ \vdots \\ \max_\pi v_\pi(s_n) \end{bmatrix}. \quad (3.6)$$

Note 3.2. BOE (3.4) 中 $p(r|s, a), p(s'|s, a)$ 作为模型部分均已知, $v(s), v(s')$ 为需求解的未知量.

3.3 Solving an Optimal Policy from the BOE

Bellman Optimality Operator

在得到 BOE (3.4) (3.5) 后自然也产生了如下问题：① **Existence & Uniqueness**: BOE 是否有解？若有解是否唯一？② **Algorithm**: 如何求解 BOE？③ **Optimality**: BOE 的解与 optimal policy 有何关系？

上述问题的一种解决角度是将 Bellman Equation (3.3) 和 Bellman Optimality Equation (3.4) (3.5) 都通过算子的角度去理解。定义一步期望奖励为 $\bar{r}(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{s', r} p(s', r | s, a)r$ ，那么式 (3.3) 可以写为：

$$v_\pi(s) = \sum_a \pi(a | s) \left[\bar{r}(s, a) + \gamma \sum_{s'} P(s' | s, a) v_\pi(s') \right] \quad (3.7)$$

也可以写成如下 Bellman Operator 形式：

$$\mathbf{v}_\pi = T^\pi \mathbf{v}_\pi, \quad (T^\pi v)(s) = \sum_a \pi(a | s) \left[\bar{r}(s, a) + \gamma \sum_{s'} P(s' | s, a) v(s') \right]. \quad (3.8)$$

Definition 3.3 (Bellman Optimality Operator). Bellman optimality operator $T^* : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$ 定义为：

$$(T^* \mathbf{v})(s) \triangleq \max_{a \in \mathcal{A}(s)} \left[\bar{r}(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v(s') \right] = \max_{a \in \mathcal{A}(s)} q_{\mathbf{v}}(s, a). \quad (3.9)$$

其中 $\bar{r}(s, a) \triangleq \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ 。从而 Bellman optimality equation (BOE) 可以写为如下不动点方程：

$$\mathbf{v}^* = T^* \mathbf{v}^*, \quad v^*(s) = \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v^*(s') \right], \forall s. \quad (3.10)$$

Existence and Uniqueness of Solution in BOE

对于不动点方程 (3.10)，根据压缩映射原理 (Contraction Mapping Theorem) / 不动点定理 (Fixed Point Theorem)，如果 T^* 是压缩映射 (见附录 B)，即存在 $c \in [0, 1)$ 使得

$$\|T^*x - T^*y\| \leq c\|x - y\|, \quad \forall x, y, \quad (3.11)$$

那么 T^* 存在唯一不动点 $x^* = T^*x^*$ ，并且从任意初值 x_0 出发，迭代 $x_{k+1} = T^*x_k$ 都收敛到 x^* 。

Theorem 3.1 (Contraction property of $f(v)$). BOE 中的映射 T^* 是一个压缩映射， $\forall \mathbf{v}, \mathbf{u} \in \mathbb{R}^{|S|}$ ，满足

$$\|T^*\mathbf{u} - T^*\mathbf{v}\|_\infty \leq \gamma \|\mathbf{u} - \mathbf{v}\|_\infty, \quad \|\mathbf{v}\|_\infty \triangleq \max_{s \in S} |v(s)|, \quad (3.12)$$

其中 $\gamma \in [0, 1)$ 为 discount factor， $\|\cdot\|_\infty$ 为 maximum norm (取绝对值最大者)。

由 Banach 压缩映射定理，可以立即得到 BOE 解的存在性、唯一性以及相应的迭代算法：

Theorem 3.2 (Existence, Uniqueness, and Algorithm). 对于 BOE $\mathbf{v}^* = T^*\mathbf{v}^* = \max_{\pi} (r_\pi + \gamma P_\pi v)$ ，总存在唯一解，且该解可以被如下方式迭代逼近：

$$\mathbf{v}_{k+1} = T^*\mathbf{v}_k = \max_{\pi} (r_\pi + \gamma P_\pi \mathbf{v}_k), \quad (3.13)$$

且产生的序列 $\{\mathbf{v}_k\}$ 指数收敛至不动点 $\mathbf{v}^*$ ，收敛速率由 γ 控制。进一步地，其误差满足

$$\|\mathbf{v}_k - \mathbf{v}^*\|_\infty \leq \gamma^k \|\mathbf{v}_0 - \mathbf{v}^*\|_\infty. \quad (3.14)$$

Note 3.3. ① **Uniqueness of optimal policy**: Optimal state value v^* 是唯一确定的，但 optimal policy 并不一定，有可能出现两个 policy 均为最优。② **Stochasticity of optimal policy**: 从 optimal policy 并不一定唯一就可以看出 optimal policy 既可以是随机的也可以是确定性的，但根据定理 3.4 可以确定的是一定存在一个确定性的 optimal policy。

Greedy Policy

Theorem 1 中虽然给出了迭代算法 (式 (3.13)) 并且给出了收敛保证, 但 $\max_{\pi}$ 尚未给出具体的选择方式.

假设环境模型已知, 即奖励概率 $p(r | s, a)$ 、状态转移概率 $p(s' | s, a)$ 、折扣因子 γ 均已知, 为了数值求解, 我们先任意初始化一个 policy value function v_0 (例如 $v_0(s) = 0, \forall s$). 此时 action value q 也会得到初始化:

$$q_0(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_0(s'). \quad (3.15)$$

由于 $v_{\pi}(s) = \sum_a \pi(a | s) q_{\pi}(s, a)$, 在 $q_{\pi}(s, a)$ 均已知的情况求解此凸组合的最大值显然有:

$$v_0(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \cdot q_0(s, a) \leq \underbrace{\left(\sum_{a \in \mathcal{A}} \pi(a | s) \right)}_{=1} \cdot q(s, a^*) = q(s, a^*), \quad q(s, a^*) = \max_{a \in \mathcal{A}(s)} q(s, a). \quad (3.16)$$

因此在当前的环境模型 (action value 分布) 下为最大化 state value, 只需要只取最大的 action value $q(s, a)$ 即可:

$$\max_{\pi} v = \max_{\pi} \sum_{a \in \mathcal{A}} \pi(a | s) q(s, a) = \max_{a \in \mathcal{A}(s)} q(s, a). \quad (3.17)$$

此时的 policy π 只选择了最大的 action value 值, 因此被称为 greedy policy:

$$\pi(a | s) = \begin{cases} 1, & a = a^* \\ 0, & a \neq a^* \end{cases}, \quad a^* = \arg \max_a q(s, a). \quad (3.18)$$

Note 3.4. Greedy policy 告诉我们: 给定 v 后最好的 one-step policy 是关于 v 的 greedy policy, 即只选择最大化 action value 的 action. 然而 greedy policy 并不能表明当前 policy 是全局最优的, 因为当前 v 可能是一个错误估计. 但 Theorem 1 给出的迭代保证说明反复应用 T^* 、按照 greedy policy 不断迭代最终必然收敛到唯一的最优解.

因此迭代公式 (3.13) 可写为:

$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_k(s') \right] = \max_{a \in \mathcal{A}(s)} q(s, a). \quad (3.19)$$

3.4 Optimality Analysis

Optimality Analysis

BOE 实际是固定了最优 policy 下的特殊 BE. Bellman Optimal Equation (3.4) (3.5) 求解的是 optimal state value v^* , 因此此时的 policy π^* 满足 Definition 3.2 中 optimal policy 的定义, 并且是一个固定的 policy, 其满足:

$$\pi^*(\cdot | s) \in \arg \max_{\pi(\cdot | s)} \sum_a \pi(a | s) \left[\bar{r}(s, a) + \gamma \sum_{s'} p(s' | s, a) v^*(s') \right]. \quad (3.20)$$

此时 Bellman Optimal Equation 在固定 policy π^* 下就可以转化为一个特殊的 Bellman Equation:

$$\mathbf{v}^* = \mathbf{r}_{\pi^*} + \gamma \mathbf{P}_{\pi^*} \mathbf{v}^* \quad (3.21)$$

关于 Bellman Optimal Equation 解的最优性, 有如下结论 (证明见附录 B).

Theorem 3.3. 通过 Bellman Optimal Equation 求解得到的 v^* 是 optimal state value, π^* 是 optimal policy, 即

$$v^* = v_{\pi^*} \geq v_{\pi}, \forall \pi. \quad (3.22)$$

Theorem 3.4 (Greedy Optimal Policy). $\forall s \in \mathcal{S}$, BOE 的最优 policy (也称为 deterministic greedy policy) 为:

$$\pi^*(a | s) = \begin{cases} 1 & \text{if } a = a^*(s) \\ 0 & \text{if } a \neq a^*(s) \end{cases} \quad (3.23a)$$

$$a^*(s) \triangleq \arg \max_a q^*(s, a), \quad q^*(s, a) \triangleq \sum_{r \in \mathcal{R}} p(r | s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v^*(s'). \quad (3.23b)$$

Analysis for Bellman Optimal Equation

对于 Bellman Optimal Equation:

$$v(s) = \max_{\pi} \sum_{a \in \mathcal{A}} \pi(a|s) \left(\sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s') \right) \quad (3.24)$$

黑色部分是未知并需求解的量，红色部分为已知量，可能对最终结果造成影响，其中

1. r : 预先设计的 reward.
2. $p(r|s, a), p(s'|s, a)$: 系统模型. 其中 $p(s' | s, a)$ 描述环境如何转移, $p(r | s, a)$ 描述环境给出多少即时奖励.
3. γ : discount factor.

由于系统的模型由外部决定，一旦确定后 agent 不能直接修改，因此主要分析 r 和 γ 的改变对 BOE 结果的影响。

Note 3.5. 策略 π 决定 agent 想做什么；系统模型 P 决定它做了以后世界实际如何演化. 例如机器人执行“向前走” (forward)，可能 $P(\text{向前移动} | s, \text{forward}) = 0.9, P(\text{停留原地} | s, \text{forward}) = 0.1$.

Discount factor determines long-sighted / short-sighted. Discounted return 的定义为

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

因此第 k 步之后得到的 reward R ，在当前时刻只值 $\gamma^k R$ ，因此 γ 越小延迟奖励衰减得越快. 因此 ① 较大的 γ 会使 agent “远视” (long-sighted)，更加重视未来的 reward. ② 相反地，较小的 γ 会使 agent “短视” (short-sighted).

Meaningless Detour. Discount factor 的存在使得一些无意义的绕远路 (meaningless detour) 策略被抑制，因为这样的 reward 会被延后且“打折”. 例如 T 步后到达终点有 return $G_{\text{direct}} = \gamma^T R$ ，但绕路后为 $G_{\text{detour}} = \gamma^{T+d} R$.

Degeneration. 将 BOE 写成如下形式:

$$v^*(s) = \max_a q^*(s, a), \quad q^*(s, a) = \bar{r}(s, a) + \gamma \sum_{s'} P(s' | s, a) v^*(s')$$

当 $\gamma = 0$ 时 BOE 退化成 $v^*(s) = \max_a \bar{r}(s, a)$. 此时未来状态的价值完全消失，因此 agent 只能最大化一步即时奖励. 这时若到达目标需要先经历若干步“即时收益低”甚至“即时惩罚”的动作 agent 就可能永远达不到目标.

Theorem 3.5 (Optimal Policy Invariance). 考虑一马尔可夫决策过程 (MDP)，其中 $\mathbf{v}^* \in \mathbb{R}^{|\mathcal{S}|}$ 是满足

$$\mathbf{v}^* = \max_{\pi \in \Pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}^*)$$

的 optimal state value. 如果对 reward $\mathbf{r} \in \mathbb{R}$ 进行仿射变换 $\alpha \mathbf{r} + \beta$ ，其中 $\alpha, \beta \in \mathbb{R}$ 且 $\alpha > 0$ ，则相应的 optimal state value $\mathbf{v}'$ 也将是 $\mathbf{v}^*$ 的仿射变换:

$$\mathbf{v}' = \alpha \mathbf{v}^* + \frac{\beta}{1 - \gamma} \mathbf{1},$$

其中 discount factor $\gamma \in (0, 1)$ ， $\mathbf{1} = [1, \dots, 1]^T$. 因此从 $\mathbf{v}'$ 导出的 optimal state value 对于 reward 的仿射变换保持不变.

3.5 Value Iteration

Value Iteration

Theorem 1 给出的迭代算法式 (3.13) 实际被称为 Value Iteration, 因为其通过不断迭代 state value v_k 逼近 v^* :

$$v_{k+1} = T^* v_k = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k). \quad (3.25)$$

实际上上述算法的实现过程可以拆解两步: ① Policy Update (PU) ② Value Update (VU).

① **Policy Update (PU)**. 在给定 v_k, r_k, P_{π} 前提下首先计算式 (3.25) 右边部分, 即估计新的 π_{k+1} :

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad \pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s') \right)}_{q_k(s, a)}, s \in \mathcal{S}. \quad (3.26)$$

根据 greedy policy (3.18), 此时的 π_{k+1} 选择最大的 action value:

$$\pi_{k+1}(a | s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg \max_a q_k(s, a). \quad (3.27)$$

② **Value Update (VU)**. 在得到 π_{k+1} 后再对 state value 进行更新:

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k, \quad v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s') \right)}_{q_k(s, a)} = \max_a q_k(s, a), s \in \mathcal{S}. \quad (3.28)$$

Note 3.6. ① v_k 是 optimal state value v^* 的中间估计, 不一定是任何策略的精确状态价值. ② q_k 是基于 v_k 的一步前瞻估计, 不一定是某个 policy 的精确 action value.

上述过程可以总结为: ① 得到 state value 中间估计 $v_k(s) \rightarrow$ ② 计算 action value 前瞻估计 $q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s') \rightarrow$ ③ greedy policy $a_k^*(s) = \arg \max_a q_k(s, a) \rightarrow$ ④ value update $v_{k+1}(s) = \max_a q_k(s, a)$.

$$v_k(s) \rightarrow q_k(s, a) \rightarrow \text{greedy policy } \pi_{k+1}(a | s) \rightarrow \text{new value } v_{k+1} = \max_a q_k(s, a)$$

Algorithm 1 Value Iteration Algorithm

Require: The probability model $p(r | s, a)$ and $p(s' | s, a)$ for all (s, a) are known. Initial guess v_0 , tolerance $\theta > 0$.

Ensure: Approximate optimal value function v^* and a greedy policy π^* .

```

1: while  $\|v_k - v_{k-1}\| \geq \theta$  do
2:   for each state  $s \in \mathcal{S}$  do
3:     for each action  $a \in \mathcal{A}(s)$  do
4:        $q_k(s, a) \leftarrow \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_k(s')$  ▷  $q$ -value
5:     end for
6:      $a_k^*(s) \leftarrow \arg \max_a q_k(s, a)$  ▷ Maximum action value
7:   Policy Update: ▷ greedy policy
```

$$\pi_{k+1}(a | s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$$

```

8:   Value Update:  $v_{k+1}(s) \leftarrow \max_a q_k(s, a)$ 
9: end for
10: end while
```

3.6 Policy Iteration

Policy Iteration

Value Iteration 的原理是聚焦于 state value，反复应用 Bellman optimality operator 不断迭代 state value v_k 逼近 v^* ，其目的是不断获取更好的 state value 中间估计 (不一定是真实的 state value)。在 Definition 3.2 中最优性同时定义了 optimal policy，因此另一个视角是聚焦于 policy，通过循环更新一个真实的 policy 并将其迭代至最优。

Policy Iteration 需要基于真实的 state value 对 policy 进行迭代，因此分为两步：① Policy Evaluation (PE)，即基于当前 policy 根据 BE 计算 state value ② Policy Improvement (PI)，即根据当前 state value 迭代 policy。

① **Policy Evaluation (PE)**. 基于当前 policy，根据 Bellman Equation 求解 policy:

$$\mathbf{v}_{\pi_k} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}, \quad v_{\pi_k}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right), s \in S, \quad (3.29)$$

其中 policy π_k 由上一步迭代得到， $\mathbf{r}_{\pi_k}, \mathbf{P}_{\pi_k}$ 均为已知系统模型， $\mathbf{v}_{\pi_k}$ 为预求未知量。

② **Policy Improvement (PI)**. 基于当前真实的 state value 使用 greedy policy 更新 policy:

$$\begin{aligned} \pi_{k+1} &= \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k}), \quad \pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s, a)}, s \in S, \\ \Rightarrow \pi_{k+1}(a | s) &= \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg \max_a q_{\pi_k}(s, a). \end{aligned} \quad (3.30)$$

在上述 Policy Iteration 框架中，实际有一些重要问题并不明晰:

1. **Q 1:** 在 **Policy Evaluation (PE)** 中如何求解 $\mathbf{v}_{\pi_k} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}$? 使用迭代法还是显示求解?

• **A 1:** 由于闭式解 $\mathbf{v}_{\pi_k} = (\mathbf{I} - \gamma \mathbf{P}_{\pi_k})^{-1} \mathbf{r}_{\pi_k}$ 中矩阵求逆数值上很困难，因此转而使用迭代法:

$$\mathbf{v}_{\pi_k}^{(j+1)} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \dots \quad (3.31)$$

2. **Q 2:** 在 **Policy Improvement(PI)** 中 policy π_{k+1} 一定比 π_k 更好吗? 为什么?

• **A 2:** 详见附录B Theorem ?? 及其证明.

3. **Q 3:** 算法最终一定收敛至 optimal policy π^* 吗? 为什么?

• **A 3:** 一定收敛至 optimal policy. 详见附录B Theorem ?? 及其证明.

Policy Iteration 算法运行过程中会先后产生如下序列:

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

注意上述 policy value 都是真实的 policy value，并非像 value iteration 一样是中间估计。

Note 3.7. ① 在迭代过程中接近目标的 state 相较于远离目标的更先达到 optimal policy，因为前者往往是后者的前提。
② 距离目标较近的 state 相较于远离目标的 state 的 state value 更大，因为较远的 state 在接近目标时获得的正奖励 (positive reward) 已经多次 discount.

Policy Iteration Algorithm

Algorithm 2 Policy Iteration Algorithm

Require: The probability model $p(r | s, a)$ and $p(s' | s, a)$ for all (s, a) are known. Initial guess π_0 , tolerance θ_1, θ_2 .

Ensure: **Aim:** Search for the optimal state value and an optimal policy.

```

1: while  $\|v_{\pi_k} - v_{\pi_{k-1}}\| \geq \theta_1$ , for the  $k$ -th iteration do                                ▷ means not converged
2:   Step 1: Policy Evaluation(PE):                                                    ▷ to calculate the state value of  $\pi_k$ 
3:   Initialization: an arbitrary initial guess  $v_{\pi_k}^{(0)}$ 
4:   while  $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\| \geq \theta_2$  do                                    ▷ means not converged
5:     for every state  $s \in S$  do                                                    ▷ iteration method
6:        $v_{\pi_k}^{(j+1)}(s) \leftarrow \sum_a \pi_k(a | s) \left[ \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_{\pi_k}^{(j)}(s') \right]$ 
7:     end for
8:   end while
9:   Step 2: Policy Improvement(PI):
10:  for every state  $s \in S$  do
11:    for every action  $a \in A(s)$  do
12:       $q_{\pi_k}(s, a) \leftarrow \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_{\pi_k}(s')$ 
13:    end for
14:     $a_k^*(s) \leftarrow \arg \max_a q_{\pi_k}(s, a)$ 
15:    Update policy:
16:       $\pi_{k+1}(a | s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$ 
17:  end for
18: end while

```

Comparison of Policy Iteration and Value Iteration

Table 3.1: Comparison of policy iteration and value iteration

Policy Iteration	Value Iteration
PE: $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$	PU: $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$
PI: $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$	VU: $v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$
$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} \dots$	$v_0 \xrightarrow{PU} \pi'_1 \xrightarrow{VU} v_1 \xrightarrow{PU} \pi'_2 \xrightarrow{VU} v_2 \xrightarrow{PU} \dots$

* 红色与蓝色部分为两种算法迭代的侧重点。

* PE: Policy evaluation; PI: Policy improvement; PU: Policy update; VU: Value update.

Table 3.2: Comparison of corresponding steps in policy iteration and value iteration

Stage	Policy Iteration	Value Iteration
1. Policy	π_0	
2. Value ^a	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 := v_{\pi_0}$
3. Policy ^b	$\pi_1 \in \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_0})$	$\pi'_1 \in \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_0)$
4. Value ^c	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi'_1} + \gamma P_{\pi'_1} v_0$
5. Policy	$\pi_2 \in \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_1})$	$\pi'_2 \in \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_1)$
	⋮	

^a For comparison, we initialize value iteration with $v_0 = v_{\pi_0}$. In general, v_0 may be arbitrary.

^b If $v_0 = v_{\pi_0}$ and the same tie-breaking rule is used, then $\pi'_1 = \pi_1$.

^c Policy iteration performs a complete policy evaluation, whereas value iteration performs only one Bellman backup.

3.7 Truncated Policy Iteration

Truncated Policy Iteration

通过 Table 3.2 可以看出, Policy Iteration 与 Value Iteration 在相同初始化的情况下第一次出现差异是在第 4 步, 此时 Policy Iteration 理论上是要完全求解出 Bellman Equation, 若使用迭代法求解即需要从 v_0 迭代 (无数次) 至收敛, 但 Value Iteration 只需要从 v_0 迭代一步. Truncated Policy Iteration 则是二者的中间方案: 计算 j 次:

$$\begin{aligned}
 v_{\pi_1}^{(0)} &= v_0 \\
 \text{value iteration} \leftarrow v_1 &\leftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
 v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
 &\vdots \\
 \text{truncated policy iteration} \leftarrow \bar{v}_1 &\leftarrow v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
 &\vdots \\
 \text{policy iteration} \leftarrow v_{\pi_1} &\leftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
 \end{aligned} \tag{3.32}$$

Note 3.8. ① 上述比较都建立于假设 $v_{\pi_1}^{(0)} = v_0 = v_{\pi_0}$, 否则两种算法不能比较. ② Policy Iteration 中实际迭代法解 BOE 时也不会进行无数次计算, 也是一个截断的. ③ Truncated Policy Iteration 在计算 v_{π_k} 时次数 j 多于 value iteration 算法但少于 policy iteration 算法. 计算次数 j 控制每轮策略评价深度与外层改进次数之间的计算权衡. 最佳 m 取决于状态规模、稀疏性、 γ 和求解器. 详细分析见附录B Theorem ?? 及其证明.

Algorithm 3 Truncated Policy Iteration Algorithm

Require: The probability model $p(r | s, a)$ and $p(s' | s, a)$ for all (s, a) are known. Initial guess π_0 .

Ensure: Search for the optimal state value and an optimal policy.

```

1: while the policy has not converged do
2:   Policy evaluation:
3:   Initialization: select the initial guess as  $v_k^{(0)} = v_{k-1}$ . The maximum iteration is set to  $j_{\text{truncate}}$ .
4:   while  $j < j_{\text{truncate}}$  do
5:     for every state  $s \in S$  do
6:        $v_k^{(j+1)}(s) \leftarrow \sum_a \pi_k(a | s) \left[ \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_k^{(j)}(s') \right]$ 
7:     end for
8:   end while
9:   Set  $v_k \leftarrow v_k^{(j_{\text{truncate}})}$ 
10:  Policy improvement:
11:  for every state  $s \in S$  do
12:    for every action  $a \in \mathcal{A}(s)$  do
13:       $q_k(s, a) \leftarrow \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_k(s')$ 
14:    end for
15:     $a_k^*(s) \leftarrow \arg \max_a q_k(s, a)$ 
16:    Update policy:
17:     $\pi_{k+1}(a | s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$ 
18:  end for

```

Small Summary

① 上述三种算法都包含两步: 1. 更新 value, 2. 更新 policy, 这种思想被称为 **generalized policy iteration**[1], 是强化学习算法中的重要思想.

② Policy Iteration 与 Value Iteration 都是 model-based reinforcement learning (MBRL) 的方法, 更准确地说是动态规划 (dynamic programming) 的方法. Policy Iteration 是下一节 Monte Carlo Learning (model-free) 的基础.